

Phase 1 — Environment & Project Foundation

LEARN

- Python installation
- VS Code
- Git
- GitHub
- Virtual Environment
- pip
- Postman

PRACTICE

- Install tools
- Create GitHub repo
- Create virtual environment
- Install dependencies

PROJECT

Create project

```
AI-Knowledge-Assistant
```

Only basic folders

```
app/  
main.py  
requirements.txt  
README.md  
.gitignore  
.env
```

PROJECT STATUS

✓ FastAPI server runs

```
Hello AI Backend
```

Phase 2 — Python Fundamentals

LEARN

- Variables
- Data Types

- Operators
- Loops
- Conditions
- Functions
- Modules
- OOP
- Exception Handling
- File Handling

PRACTICE

- Calculator
- Library Management
- Mini Programs

PROJECT

Instead of APIs,

build backend utilities.

Create

```
config.py
```

```
utils.py
```

```
logger.py
```

Project becomes organized.

Phase 3 — REST API

LEARN

- HTTP
- REST
- JSON
- Requests
- Responses
- Headers
- Status Codes

PRACTICE

Test APIs using Postman.

PROJECT

Create

```
GET /
```

```
GET /health
```

```
GET /version
```

Now backend can receive requests.

Swagger works.

Phase 4 — FastAPI

LEARN

- FastAPI
- Routing
- Pydantic
- Dependency Injection
- Validation

PRACTICE

CRUD examples

PROJECT

Create

```
User APIs
```

```
Create User
```

```
Get User
```

```
Update User
```

```
Delete User
```

Currently store data in memory.

Project is now a proper backend.

Phase 5 — Database

LEARN

- PostgreSQL
- SQL
- SQLAlchemy
- ORM
- Relationships
- CRUD

PRACTICE

Create database

Create tables

PROJECT

Replace

Python List

with

PostgreSQL

Tables

Users

Chats

Documents

Now data is persistent.

Phase 6 — Authentication

LEARN

- JWT
- Access Token
- Password Hashing
- Authorization
- RBAC

PRACTICE

Login API

PROJECT

Add

Register
Login
Protected Routes
Current User

Backend becomes secure.

Phase 7 — LLM

LEARN

- Generative AI
- OpenAI API
- Prompt Engineering
- Tokens

PRACTICE

Prompt experiments

PROJECT

Create

POST /chat

User asks

Hello

LLM replies.

Your backend becomes AI-powered.

Phase 8 — LangChain

LEARN

- PromptTemplate
- Chains
- Memory

- Output Parser

PRACTICE

Simple chains

PROJECT

Replace

OpenAI **API**

with

LangChain

Now chatbot remembers previous conversation.

Phase 9 — Vector Database

LEARN

- Embeddings
- ChromaDB
- Similarity Search

PRACTICE

Store vectors

Search vectors

PROJECT

User uploads PDF.

Backend creates embeddings.

Stores them inside ChromaDB.

Phase 10 — RAG

LEARN

- Document Loader
- Chunking
- Retriever
- Context
- RAG Pipeline

PRACTICE

PDF Question Answering

PROJECT

Ask

Summarize Chapter 2

Backend searches vector database.

Returns answer using document context.

Project becomes a real RAG system.

Phase 11 — Backend Architecture

LEARN

- Microservices
- Redis
- RabbitMQ
- Async
- Caching

PRACTICE

Redis demo

RabbitMQ basics

PROJECT

Cache repeated AI responses.

(Optional)

Generate embeddings in background.

Project becomes faster and more scalable.

Phase 12 — DevOps

LEARN

- Docker
- Docker Compose
- GitHub Actions
- CI/CD
- Environment Variables

PRACTICE

Containerize application

PROJECT

Run

```
docker compose up
```

Everything starts with one command.

CI pipeline builds automatically.

Phase 13 — Testing

LEARN

- Pytest
- API Testing
- Unit Testing
- Mocking

PRACTICE

Write tests

PROJECT

Test

- Authentication
- Users
- Chat
- PDF Upload
- RAG

Project becomes reliable.

Phase 14 — Production Ready

LEARN

Backend best practices

PROJECT

Improve

- Logging
- Error Handling
- Validation
- Folder Structure
- README
- Environment Variables

- API Documentation

Project now looks like an industry project.

Phase 15 — Review & Demo

FINAL TASKS

- Code Review
- Bug Fixes
- Documentation
- GitHub Cleanup
- Demo

Release

AI Knowledge Assistant v1.0

Final Project Structure (End State)

```
AI-Knowledge-Assistant/  
|  
├─ app/  
|   └─ api/  
|       ├── auth.py  
|       ├── users.py  
|       ├── chat.py  
|       ├── documents.py  
|       └─ health.py  
|   |  
|   └─ core/  
|       ├── config.py  
|       ├── security.py  
|       └─ logger.py  
|   |  
|   └─ database/  
|       ├── connection.py  
|       └─ models.py
```